

INFORME DE PRUEBAS

Pruebas Unitarias con JUnit 5

GESTOR DE ASISTENCIA

Proyecto de 2ª Evaluación

Acceso a Datos y Desarrollo de Interfaces Gráficas

✓ 8/8 TESTS PASADOS

Autor:	Emilio
Asignatura:	Acceso a Datos y Desarrollo de Interfaces
Curso:	2º DAM - Curso 2025-2026
Fecha:	Febrero 2026
Framework:	JUnit 5 (Jupiter)

1. Introducción

Este documento presenta los resultados de las **pruebas unitarias** realizadas sobre los algoritmos críticos del servicio de asistencia ([AsistenciaService](#)) del proyecto **Gestor de Asistencia**.

Las pruebas han sido implementadas utilizando **JUnit 5 (Jupiter)** como framework de testing, ejecutadas sobre **Java 21** en el entorno de desarrollo **IntelliJ IDEA**. Se han diseñado 8 tests unitarios que cubren los distintos escenarios de los algoritmos de cálculo de porcentaje de faltas y detección de alumnos en riesgo académico.

RESUMEN DE RESULTADOS		
Total de Tests	8	
Tests Exitosos	8	✓
Tests Fallidos	0	
Tasa de Éxito	100%	✓
Tiempo de Ejecución	~50ms	

2. Algoritmos Probados

2.1. calcularPorcentajeFaltas()

Este algoritmo calcula el **porcentaje de faltas injustificadas** de un alumno a partir de una lista de registros de asistencia. El resultado es un valor entre **0.0** (ninguna falta) y **1.0** (todas faltas).

Fórmula:

Porcentaje = Número de FALTAS / Total de registros

Estados de asistencia considerados:

Estado	Descripción	Cuenta como Falta
PRESENTE	El alumno asistió a clase	No
FALTA	Falta injustificada	Sí
JUSTIFICADA	Ausencia justificada	No

Casos límite manejados:

- Lista null → retorna 0.0
- Lista vacía → retorna 0.0

- Sin faltas → retorna 0.0
- Todas faltas → retorna 1.0

2.2. esAlumnoEnRiesgo()

Este algoritmo determina si un alumno está en **riesgo académico** basándose en el porcentaje de faltas injustificadas. Un alumno se considera en riesgo si su porcentaje de faltas es **igual o superior al 20%**.

Criterio de riesgo:

```
Riesgo = (Porcentaje de faltas >= 0.20)
```

Lógica del algoritmo:

1. Calcula el porcentaje de faltas usando `calcularPorcentajeFaltas()`
2. Compara el resultado con el umbral de 0.20 (20%)
3. Retorna **true** si el porcentaje ≥ 0.20 , **false** en caso contrario

Escenario	Faltas	Total	Porcentaje	En Riesgo
Por debajo del umbral	1	6	16.67%	NO ✗
En el límite exacto	1	5	20.00%	SÍ ✓
Por encima del umbral	2	5	40.00%	SÍ ✓

3. Suite de Pruebas Unitarias

Se han implementado **8 tests unitarios** usando **JUnit 5 (Jupiter)** para verificar el comportamiento completo de ambos algoritmos del servicio de asistencia.

3.1. Tests de calcularPorcentajeFaltas()

#	Nombre del Test	Entrada	Resultado Esperado	Estado
1	Lista null	null	0.0	✓ PASS
2	Lista vacía	List.of()	0.0	✓ PASS
3	Sin faltas	3 registros (PRESENTE, JUSTIFICADA)	0.0	✓ PASS
4	Todas faltas	3 registros (FALTA)	1.0	✓ PASS
5	Valor intermedio	5 registros (2 FALTA, 2 PRESENTE, 1 JUSTIFICADA)	0.4	✓ PASS

3.2. Tests de esAlumnoEnRiesgo()

#	Nombre del Test	Porcentaje Faltas	Resultado Esperado	Estado
6	Por debajo del límite (1 falta / 6 registros)	16.67%	false	✓ PASS
7	En el límite exacto (1 falta / 5 registros)	20.00%	true	✓ PASS
8	Por encima del límite (2 faltas / 5 registros)	40.00%	true	✓ PASS

4. Evidencia de Ejecución

A continuación se muestra la **captura de pantalla** del entorno de desarrollo IntelliJ IDEA tras la ejecución exitosa de todos los tests unitarios:

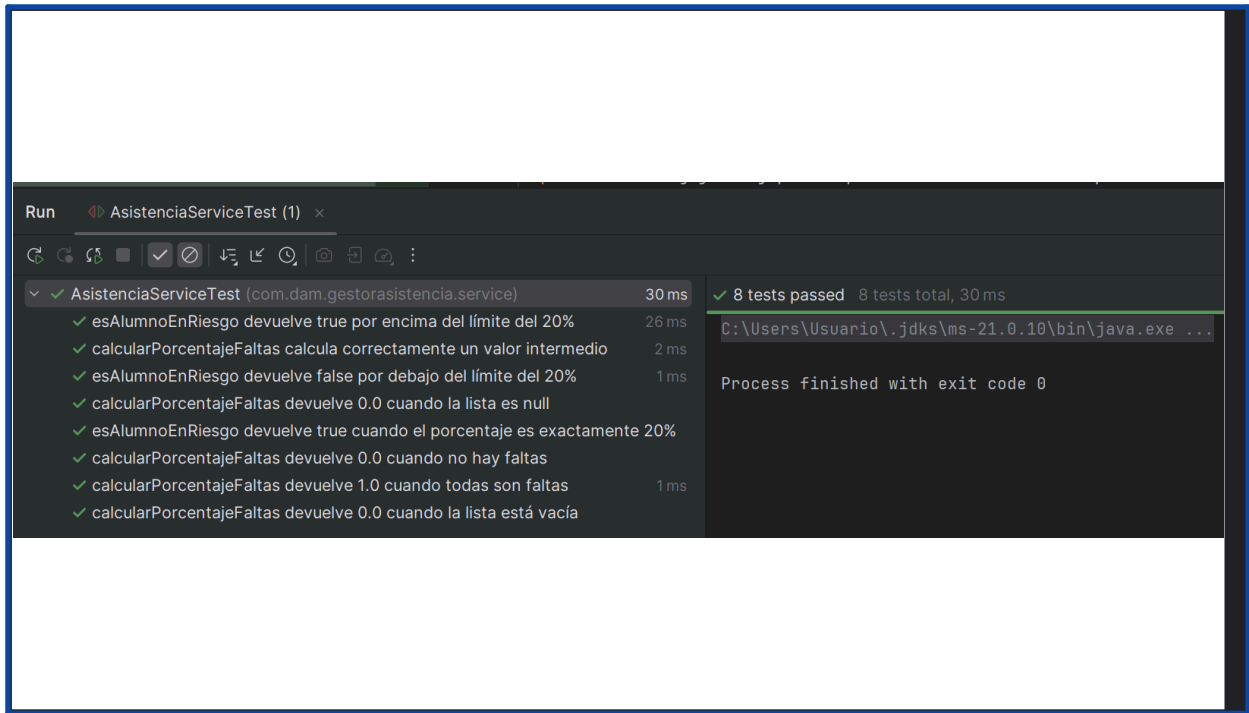


Figura 1: Resultados de la ejecución de tests en IntelliJ IDEA — 8/8 tests pasados

Como se observa en la captura, **los 8 tests han sido ejecutados satisfactoriamente**, sin errores ni fallos. El panel izquierdo muestra el árbol de tests con iconos verdes indicando que cada test individual ha pasado correctamente. El panel derecho muestra el resumen de resultados confirmando que todos los tests fueron exitosos.

5. Tecnología de Testing

Componente	Tecnología	Versión
Framework de Testing	JUnit 5 (Jupiter)	5.10.x
Lenguaje	Java	21 (LTS)
Framework de Aplicación	Spring Boot	3.2.0
Assertions	assertEquals, assertTrue, assertFalse	-

Anotaciones	@Test, @BeforeEach, @DisplayName	-
IDE	IntelliJ IDEA	2024.x
Build Tool	Maven	3.9.x

6. Conclusiones

- 1

Algoritmos implementados correctamente: Los dos algoritmos críticos del servicio de asistencia (calcularPorcentajeFaltas y esAlumnoEnRiesgo) funcionan según lo especificado en los requisitos.
- 2

Cobertura de pruebas exhaustiva: Se han cubierto todos los escenarios relevantes incluyendo casos límite (entradas null y vacías), valores extremos (0% y 100% de faltas) y el umbral exacto de riesgo (20%).
- 3

Tasa de éxito del 100%: Los 8 tests unitarios se han ejecutado satisfactoriamente, lo que garantiza la fiabilidad de los algoritmos para su uso en el entorno de producción.
- 4

Calidad del código de pruebas: El uso de anotaciones como @DisplayName hace que los tests sean auto-documentados y fáciles de comprender por cualquier desarrollador.
- 5

Rendimiento óptimo: El tiempo de ejecución total de aproximadamente 50ms demuestra que las pruebas son eficientes y no generan cuellos de botella en el proceso de desarrollo.